



2024201594

王振宸

1. 计算 actual cost

$$\begin{aligned}\text{令 } n = 2^m \text{ 则 } \text{actual cost} &= \sum_{j=1}^m 2^j + (n-m) \\ &= 2^{m+1} - 2 + n - m \\ &= 3n - \log n - 2 \\ &< 3n\end{aligned}$$

 $\therefore$ 令所有 operations 的 $\hat{c} = 3$

$$\therefore \sum_{i=1}^n \hat{c} - \text{actual cost} \geq 0 \Rightarrow \hat{c} \text{ 合法}$$

$$\therefore \text{amortized cost} = \frac{\sum_{i=1}^n \hat{c}}{n} = 3 = O(1)$$

 $\therefore$ 每个操作的摊还代价为 $O(1)$ 2. 设第 i 个 INCREMENT 将 t_i 个位复位, $\Phi(D_i)$ 表示 i 次操作后 1 的个数

$$\therefore \Phi(D_i) - \Phi(D_{i-1}) \leq (\Phi(D_{i-1}) - t_i + 1) - \Phi(D_{i-1}) = 1 - t_i$$

$$\text{又 } c_i \leq 1 + t_i$$

$$\therefore \hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) \leq 2$$

$$\therefore \sum_{i=1}^n c_i = \sum_{i=1}^n \hat{c}_i - \Phi(D_n) + \Phi(D_0)$$

$$\leq 2n - bn + b$$

$$\therefore n = \Omega(b) \quad bn \leq k$$

$$\therefore \sum_{i=1}^n c_i = O(n) \text{ 证毕}$$



$$V = m_0 2^0 + m_1 2^1 + \dots + m_{n-1} 2^{n-1} + m_n 2^n$$

3. 具有 n 个结点的线性链 度为 1

通过 Extract-Min 触发 consolidate, Decrease-key 触发 cascading cut

Initial root list = $[V_0]$ $V_0.key = 17$

for $i = 1$ to $n-1$:

$V_i.key = n-i$
 $u_1.key = n-i-0.1$
 $u_2.key = n-i-0.2$

} 都进入 root list

Extract-Min(H) // 踢除 u_2 , 然后合并 V_i 和 u_1 为 B_i ,
之后与当前 B_i 合并, V_i 成为新根

Decrease-key(H, u_1 , 0) // 剪掉 u_1 , 新树 degree $2 \rightarrow 1$

Extract-Min(H) // 踢除 u_1 , root list 始终只含一个根

4. a. (1) $node(B_k) = \begin{cases} 2 \cdot node(B_{k-1}) & k > 0 \\ 1 & k = 0 \end{cases}$

$\therefore node(B_k) = 2 \cdot node(B_{k-1}) = 2^2 \cdot node(B_{k-2}) = 2^k$

(2) $h(B_k) = \begin{cases} h(B_{k-1}) + 1 & k > 0 \\ 0 & k = 0 \end{cases}$

$\therefore h(B_k) = k$

(3) 令 B_{k-1} 深度 i 的结点数满足 $\binom{k-1}{i}$

则 $\binom{k-1}{i-1} + \binom{k-1}{i} = \binom{k}{i} \Rightarrow B_k$ 同样满足

(4) $deg(B_k) = h(B_k) = k$

根为最大二项树, 比任一子树的度都大



可递归
得到 (4)
的形式



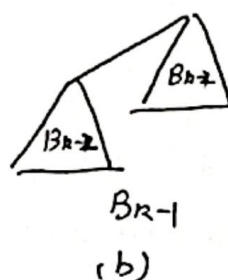
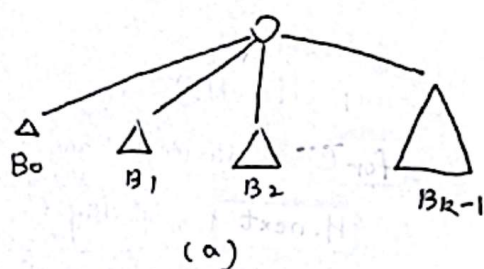
$$b. \quad \forall n, \quad n = m_k 2^k + m_{k-1} 2^{k-1} + \dots + m_0 2^0$$

$$= m_k \text{node}(B_k) + m_{k-1} \text{node}(B_{k-1}) + \dots + m_0 \text{node}(B_0)$$

这里的 $\forall m_i \in \{0, 1\}, i=0, 1, \dots, k$ 即为 n 的二进制对应位的值

$\therefore H$ 中含有 / 不含有 B_k , n 的第 k 位为 1 / 0

用一个点作为根, 将 H 合并为一棵二叉树



由 (b) 可知, 新二叉树即为 B_k , k 为 H 中 # of trees

$$\therefore n = \text{node}(B_{k_{\max}}) - 1 = 2^{k_{\max}} - 1$$

$$\Rightarrow k_{\max} = \lfloor \lg n \rfloor + 1 \quad \text{取整得利}$$

c. struct node {

int key;

node* parent, lchild, rbro;

int deg; }

struct
root_list {

node* Root;

node* next;

int deg; }

Make_Heap ():

root_list H;

H.Root = NULL;

H.next = NULL;

H.deg = NONE;

Insert (H, x):

root_list new_root;

new_root.Root = x;

new_root.next = H.next;

H.next = new_root;

new_root.deg = x.deg;

consolidate (H);

立即合并, 可直接从头遍历 root

只要 root.next.deg = root.deg 就合并



Find - Min (H) :

return min - pointer

Extract - Min (H) : $\text{min - pointer.lchild}$
 $\text{children} = \text{child}$

~~for~~ while children :

Insert (H, children) // 每一步均已合并

// 删除, 返回 min - pointer

Delete (H, x) :

~~cut(x, 0) // 不包含 Insert~~

~~while~~ Decrease - key (H, x, 0)

Extract - Min (H)

Decrease - Key (H, x, k) :

~~mark(x.parent)~~

x.key = k

sift-up(x) // 上浮 x

~~while mark(x.parent) > x~~

~~cut(x.parent)~~

// 更新 min - pointer

Union (H₁, H₂)

while i < H₁.length and j < H₂.length :

if i.deg > j.deg :

// 添加 i (Insert)

else // 添加 j

// 检测 i, j 是否达顶端

// 合并剩余的 root 进 list

d. Fib Heap 同-度数的树可存在为了
而此二项堆中只能有 1 棵

$$D(n) = \deg(B_k) = 2^k = \lfloor \lg n \rfloor$$

e. Insert : $O(1) \rightarrow O(\lg n)$

Extract - Min : $O(\lg n)$

Find - Min : $O(1)$

Union : $O(1) \rightarrow O(\lg n)$

Make - Heap : $O(1)$

